

Agentic Development Principles

Adapted from NautilusTRX fifth-pass project materials authored by Jorge Valenzuela for CIS 400 in spring 2026.

A collection of repeatable and teachable principles that can be applied uniformly across a project to get high-quality output.

Each principle answers the following questions in order: - What is it? - How to apply it? - Why apply it? - When should it be applied?

Spec Driven Development

- Using an additional document as reference material (the SPEC) for the Agent during development.
- Write a SPEC yourself, prompt the Agent to generate one. A Concept of Operations works the same.
- It provides a single source of truth for the project (no matter the scale). This makes it easy for the Agent to answer its own questions as it knows exactly where to look.
- Any medium to large projects or complex tasks. When applying this principle, the writing of the spec occurs before the first prompt of development.

The Cycle of Development

- The repeated ordered actions the Agent and you take while developing. Here is a simple example:
 1. Write Unit tests from the SPEC
 2. Develop the Code
 3. Verify Code with Unit tests
 4. Provide Checklist of work to User Approval
 5. Commit and Push
- Use plan mode to setup the Cycle of Development. Prepared Cycles of Development can be placed into Skills for easy access and to be uniformly applied across sessions and projects.
- It helps enforce the application of other development principles by providing structure for the Agent to follow.
- Any task should consider this principle, large or small. When working on a multi-phased project, writing the Cycle of Development down (in plan mode or CLAUDE.md) is essential.

Project Context Management (CLAUDE.md)

- The practice of maintaining a collection of project details, preferences and unique practices in a context file (CLAUDE.md).
- Setup your workspace (/init) with a context file, then amend it before and during development as needed.
- Unique practices, atypical conventions and project preferences should not have to be a part of your prompt. Place them in your context file (CLAUDE.md) instead.
- Management of context is an ongoing practice no matter the scale of your prompt. Consider amending your context file (CLAUDE.md) when you find yourself writing duplicate instructions across prompts, or the Agent makes the same mistake multiple times.

Requirement Elicitation

- The practice of finding and filling gaps in a specification you have made.
- The plan mode helps a lot, but plan mode alone is not sufficient. /grill-me skill or equivalent in the prompt.
- By exposing unspecified areas of your SPEC you improve the clarity of the Agent's task and in turn, create a better end result.

- Most prompts should have this. For the cost of a couple hundred tokens, you can save potential tens of thousands of tokens having to redevelop a section of code.

Verification

- The practice of ensuring what you instruct to happen actually happens.
- Both manual and Agentic version of verification Exists. Ex: Using a Code Review Subagent, developing with Unit test/testing tools, manually reviewing code yourself.
- As a developer, you are responsible for every line of code you produce with the Agent. It is vital you know that the Agent wrote exactly what it was instructed to.
- Every time the Agent writes something verification should be applied. For larger projects, it can be useful to add verification into your development cycle.